

Caso de Teste 01 - Lighthouse Desktop (BTCheck ID: TC-LH-001)

Objetivo: verificar qualidade não-funcional da home via Lighthouse (Desktop e Mobile) e capturar métricas/audits críticos para regressão.

Alvo: <https://guibim.github.io/btcheck-site/>

Pré-condições

- Site acessível.
- Lighthouse (DevTools ou CLI) disponível.
- Modo **Desktop/Mobile**.

Passos

1. Abrir a URL.
2. Rodar Lighthouse em **Desktop e Mobile** com categorias: **Performance, Accessibility, Best Practices, SEO**.
3. Salvar o relatório (HTML/JSON ou print).
4. Registrar:
 - Scores das 4 categorias.
 - Métricas: **FCP, LCP, TBT, CLS**.
5. Conferir presença/estado dos principais audits do relatório:
 - **Render-blocking resources**
 - **Long cache TTL**
 - **Unused JavaScript**
 - **Layout shift elements**
 - **Missing source maps**

FCP (First Contentful Paint): tempo até o navegador renderizar o primeiro conteúdo visível na tela (texto, imagem, SVG/canvas). Indica a “primeira impressão” de carregamento.

LCP (Largest Contentful Paint): tempo até renderizar o maior elemento de conteúdo visível na área inicial (geralmente um banner, imagem grande ou bloco principal de texto). É um dos principais indicadores de velocidade percebida.

TBT (Total Blocking Time): soma do tempo em que a main thread fica bloqueada por tarefas longas de JavaScript (normalmente > 50 ms), entre o FCP e o momento em que a página fica mais interativa. Indica o quanto o JS atrapalha cliques/inputs.

CLS (Cumulative Layout Shift): medida do quanto a página “pula”/muda de layout inesperadamente durante o carregamento (ex.: imagens sem dimensão, fontes carregando e empurrando conteúdo). Quanto menor, mais estável e confortável é a leitura/navegação.

Resultados (Lighthouse — Desktop / BTCheck)

- Performance: 89
- Acessibilidade: 88
- Best Practices: 100
- SEO: 100

Métricas

- FCP: 0,7 s
- LCP: 1,2 s
- Speed Index: 0,9 s
- TBT: 110 ms
- CLS: 0,163

Resultados (Lighthouse — Mobile / BTCheck)

- Performance: 55
- Acessibilidade: 88
- Best Practices: 100
- SEO: 100

Métricas (Mobile)

- FCP: 3,5 s
- LCP: 4,7 s
- TBT: 90 ms
- CLS: 0,427
- Speed Index: 3,5 s

Fix para v2:

Performance (prioridade máxima)

Eliminar render-blocking requests (CSS/JS no head)

Inlining de CSS crítico; defer/async em scripts; dividir CSS por rota; evitar carregamento de libs antes do first paint.

Configurar cache eficiente para assets estáticos (cache lifetimes)

Cache-Control: public, max-age=31536000, immutable em JS/CSS/imagens/fontes + filenames com hash (cache busting).

Reduzir LCP (4.7s)

Otimizar o “hero”/maior elemento acima da dobra: compressão e formato (WebP/AVIF), preload do recurso LCP, priorização de imagem (fetchpriority="high"), evitar dependência de JS para renderizar o conteúdo principal.

Reduzir FCP (3.5s) / Speed Index (3.5s)

Diminuir payload inicial: cortar JS/CSS desnecessário, lazy-load do que é abaixo da dobra, minimizar fontes e evitar FOIT.

Corrigir CLS alto (0.427)

Definir width/height (ou aspect-ratio) em imagens/iframes; reservar espaço para componentes; evitar inserir conteúdo acima da dobra; usar font-display: swap + preload de fontes críticas; garantir que banners/cards não mudem altura após load.

Remover “forced reflow”

Evitar leituras/escritas intercaladas no layout (ex.: medir offsetHeight e logo depois aplicar style repetidamente); agrupar DOM writes; usar requestAnimationFrame; preferir transform/opacity a propriedades que recalculam layout.

Reduzir JS não utilizado (~255 KiB)

Tree-shaking efetivo; code-splitting por rota; remover dependências; importar componentes sob demanda; evitar bundles “monolíticos”.

Evitar long main-thread tasks

Quebrar tarefas pesadas; lazy-load de módulos; mover cálculos para Web Worker quando fizer sentido; reduzir parsing/exec de JS no carregamento.

Evitar animações não compostas (non-composited animations)

Trocar animações de top/left/width/height por transform/opacity; garantir will-change com parcimônia; evitar animar layout.

Reduzir “network dependency tree”

Menos encadeamento de requests: preconnect/dns-prefetch para origens críticas; consolidar/adiar chamadas; reduzir redirects; priorizar recursos essenciais.

Acessibilidade (para subir de 88)

Dar nome acessível aos botões

Texto visível, aria-label, aria-labelledby ou conteúdo textual interno consistente.

Associar <label> aos <select>

label for="id" + id no select (ou aria-label/aria-labelledby quando não houver label visual).

Links distinguíveis sem depender só de cor

Adicionar sublinhado/estilo adicional (ex.: text-decoration), e garantir contraste adequado no estado normal/hover/focus.

Best Practices (já 100, mas dá para melhorar qualidade técnica)

Gerar e publicar source maps para JS first-party grande

Habilitar sourcemap no build (ou condicionar por ambiente), garantindo que não exponha segredos e que ajude debug/perf analysis.

SEO (já 100)

(Manual check) Validar structured data em ferramenta externa

Rodar validação de schema/structured data para confirmar ausência de warnings/erros fora do Lighthouse.